

Solaris

Backend API guide for Django developers moving to Java and Spring Boot

38	127 / 127	0	44 / 44
unique API routes	live HTTP checks passed	live 5xx responses	automated tests passed

What this guide answers

- What each endpoint does, who may call it, and why it exists.
- How a request travels through Spring Security, MVC controllers, services, repositories, Hibernate/JPA, JDBC, PostgreSQL, and Redis.
- Where each behavior is implemented, using repository-relative Java file paths and exact line ranges.
- How Django and Django REST Framework concepts map to Spring Boot concepts without pretending the frameworks are identical.

Line numbers refer to the Java source snapshot tested on 2026-10-01. They are navigation aids and may move after later edits.

Solar Energy Monitoring System

Java 25 - Spring Boot 4.1 - Spring Security - JPA/Hibernate - PostgreSQL - Redis - JWT - Gradle

1 Verification status

The running application was tested over real HTTP on localhost against the Docker PostgreSQL and Redis services. The recorded run contains **127 checks**, **127 passes**, **0 failures**, and **0 5xx responses**. Fixtures were uniquely named, settings were restored, and database plus Redis cleanup was verified.

Area	What was proven live
Authentication	Register, login, expired/invalid JWT, access-vs-refresh separation, rotation, replay rejection, logout, disabled/deleted users.
Authorization	Missing auth 401, wrong role 403, homeowner ownership isolation returning 404 for foreign resources.
Admin CRUD	Users, sites, devices, batteries, settings, alerts, duplicates, invalid enums, missing IDs, dependency conflicts, safe deletes.
Homeowner data	Profile, sites, energy, storage, alerts, JDBC report, empty windows, date limits, pagination limits.
Runtime	Spring Boot startup, Flyway validation, PostgreSQL access, Redis access, clean server logs, clean Gradle test and build.

No finite test suite proves every imaginable future failure, but this run covers every current route and the security, validation, ownership, conflict, empty-result, and boundary behaviors that are most likely to cause production defects.

2 Django-to-Spring mental model

Django / DRF	Solaris Spring Boot	What changes in your thinking
urls.py	@RequestMapping + @Get/Post/Put/DeleteMapping	Routes live beside controller methods rather than in one URL table.
APIView / ViewSet	@RestController	Controller receives HTTP input and delegates; it should not own business rules.
Serializer	Request/response DTO + Bean Validation	DTOs define the wire contract; @Valid triggers validation before service execution.
Model	@Entity	JPA maps Java objects and relationships to relational rows.
Manager / QuerySet	Repository	Spring Data derives queries from method names; custom JPQL/JDBC is used when clearer.
Service module	@Service	Transactions and business rules live here, explicitly separated from controllers.
Middleware / authentication class	Servlet Filter + SecurityFilterChain	JWT authentication happens before MVC chooses a controller.
permission_classes	Route authorization + ownership queries	Roles gate route families; repository predicates enforce object ownership.
transaction.atomic	@Transactional	Spring opens, commits, or rolls back the transaction around the service method.
DRF exception handler	@RestControllerAdvice	Exceptions are translated into one JSON error shape.
Celery beat task	@Scheduled Spring component	A managed scheduler triggers monitoring without manual threads.
cache backend	Redis StringRedisTemplate	Redis stores revocable refresh-session state, not domain records.

Important difference: role checks and object ownership are separate concerns. `hasRole('HOMEOWNER')` answers 'what kind of user is this?'; `findByOwnerId` answers 'does this exact row belong to this user?'. Solaris applies both.

3 How one request travels

1	Tomcat servlet container Accepts HTTP and creates <code>HttpServletRequest</code> / <code>HttpServletResponse</code> .
2	Spring Security filter chain <code>JwtAuthenticationFilter</code> validates Bearer token, token type, expiry, and current database user state.
3	Authorization <code>SecurityConfig</code> permits public auth routes and enforces ADMIN or HOMEOWNER on route families.
4	DispatcherServlet Spring MVC matches method + path to a <code>@RestController</code> method and converts JSON into a DTO.
5	Bean Validation <code>@Valid</code> and constraint annotations reject bad input before business logic.
6	Service Business rules, ownership checks, and <code>@Transactional</code> boundaries execute.
7	Repository JPA/Hibernate or parameterized JDBC talks to PostgreSQL.
8	Response DTO becomes JSON; exceptions become <code>ApiErrorResponse</code> through <code>GlobalExceptionHandler</code> .

Core navigation references

```
config/SecurityConfig.java:31-90
security/JwtAuthenticationFilter.java:34-103
exception/GlobalExceptionHandler.java:27-156
```

4 Authentication, JWT, and Redis

Access token

Short-lived JWT used only in Authorization: Bearer. It contains subject email, role, userId, token type ACCESS, issued-at, and expiry. The filter verifies the signature and expiry, rejects any non-ACCESS token, then loads the current user from PostgreSQL. That database check immediately invalidates access for deleted, disabled, email-changed, or role-changed users.

Refresh token

Longer-lived JWT used only by refresh/logout. It contains a unique jti and type REFRESH. Signature alone is insufficient: Redis must also contain an active record for that jti.

Why Redis exists

Pure JWT refresh tokens cannot be revoked before expiry. Solaris stores one short record per refresh token: key solaris:refresh:<jti>, value userId:SHA256(token), with Redis TTL equal to remaining JWT lifetime. The raw token is never stored.

Login	Issue access + refresh; hash refresh; store jti state in Redis with TTL.
Refresh	Verify JWT type/signature/expiry; atomically compare-and-delete Redis state; verify live user; issue a new pair.
Replay	Old jti no longer exists, so the second use returns 401.
Logout	Atomically consumes the Redis record, revoking future refresh.
Redis unavailable	Return 503 rather than silently accepting an unverifiable refresh token.

```
security/JwtService.java:37-67
security/RedisRefreshTokenStore.java:13-53
service/AuthService.java:54-137
security/JwtAuthenticationFilter.java:34-103
```

5 Domain model and ownership

Entity	Purpose	Relationship
User	ADMIN or HOMEOWNER account	Owns sites; receives alerts
SolarSite	Physical installation and ownership boundary	belongs to User
Device	Inverter, meter, controller, or sensor	belongs to SolarSite
Battery	Current storage asset snapshot	belongs to SolarSite
EnergyReading	Production/consumption/import/export event	belongs to Site + Device
StorageReading	Charge/stored-energy event	belongs to Site + Battery
Alert	Persisted event	belongs to User; optional Site/Device/Battery
SystemSetting	Singleton alert rule configuration	not an alert event

The most important security rule is that homeowner queries travel through the ownership chain in SQL: `site.owner.id`, `battery.site.owner.id`, `reading.site.owner.id`, or `alert.user.id`. Returning 404 for a foreign ID prevents an attacker from confirming that another user's resource exists.

6 Authentication

POST /api/auth/register

Access	Public
Purpose	Create a self-service homeowner account. This is the Spring equivalent of a DRF registration view or serializer create method.
Input	JSON: name, email, password. Name is 2-120 chars; password is 8-72 chars; email must be valid.
Success	201 Created with no response body.
Rules / edge behavior	Email is normalized to lowercase; password is BCrypt-hashed; role is always HOMEOWNER; duplicate email is 409.
Java implementation	controller/AuthController.java:20-27; service/AuthService.java:37-52; dto/auth/RegisterRequest.java:10-22; repository/UserRepository.java:16-18

POST /api/auth/login

Access	Public
Purpose	Verify credentials and issue an access/refresh token pair.
Input	JSON: email, password.
Success	200 OK with accessToken, refreshToken, expiry seconds, user identity, and role.
Rules / edge behavior	Disabled users and bad credentials both return 401 without revealing which check failed. The refresh token state is stored in Redis.
Java implementation	controller/AuthController.java:29-36; service/AuthService.java:54-64,89-110; security/JwtService.java:37-59; security/RedisRefreshTokenStore.java:24-31

POST /api/auth/refresh

Access	Public endpoint, valid REFRESH token required in body
Purpose	Rotate a refresh token and obtain a new token pair without resending the password.
Input	JSON: refreshToken.
Success	200 OK with a new access token and a new refresh token.
Rules / edge behavior	Token must be signed, unexpired, type REFRESH, present in Redis, and tied to an existing enabled user. Redis atomically consumes it, so replay returns 401.
Java implementation	controller/AuthController.java:38-42; service/AuthService.java:66-80,89-137; security/RedisRefreshTokenStore.java:17-20,33-45; dto/auth/RefreshTokenRequest.java:9-12

POST /api/auth/logout

Access	Public endpoint, valid active REFRESH token required in body
Purpose	Revoke the browser or mobile client's refresh session.
Input	JSON: refreshToken.
Success	204 No Content.
Rules / edge behavior	The Redis record is atomically consumed. A revoked, rotated, malformed, or already-used token returns 401. Existing short-lived access tokens expire naturally.
Java implementation	controller/AuthController.java:44-48; service/AuthService.java:82-87,112-137; security/RedisRefreshTokenStore.java:33-45

7 Admin user management

GET /api/admin/users

Access	ADMIN bearer token
Purpose	List accounts for administration without exposing password hashes.
Input	Query: page (default 0), size (default 20, max 100).
Success	200 OK with PageResponse.
Rules / edge behavior	Negative page or size outside 1-100 returns 400. Results sort newest first.
Java implementation	controller/AdminUserController.java:26-31; service/AdminUserService.java:26-30; dto/PageResponse.java:5-28

GET /api/admin/users/{id}

Access	ADMIN bearer token
Purpose	Inspect a single account for support or administration.
Input	Path: positive numeric user ID.
Success	200 OK with user metadata.
Rules / edge behavior	Missing user is 404; password never appears in the DTO.
Java implementation	controller/AdminUserController.java:38-45; service/AdminUserService.java:32-36; dto/user/AdminUserResponse.java:9-27

POST /api/admin/users

Access	ADMIN bearer token
Purpose	Create a controlled HOMEOWNER or ADMIN account.
Input	JSON: name, email, password, role, optional phone, enabled.
Success	201 Created with the safe user DTO.
Rules / edge behavior	Role must be ADMIN or HOMEOWNER; email is unique; password is BCrypt-hashed. This is the controlled path for ADMIN creation.
Java implementation	controller/AdminUserController.java:33-36; service/AdminUserService.java:38-52; dto/user/CreateUserRequest.java:13-34

PUT /api/admin/users/{id}

Access	ADMIN bearer token
Purpose	Change identity, role, enabled status, phone, and optionally password.
Input	JSON: name, email, role, enabled, optional phone; omit password or use null to preserve it.
Success	200 OK with updated user.
Rules / edge behavior	Duplicate email is 409. The last enabled ADMIN cannot be demoted or disabled. A supplied password must be 8-72 chars.
Java implementation	controller/AdminUserController.java:47-55; service/AdminUserService.java:55-84, 99-106; dto/user/UpdateUserRequest.java:13-34

DELETE /api/admin/users/{id}

Access	ADMIN bearer token
Purpose	Remove an account when business dependencies permit it.
Input	Path: user ID.
Success	204 No Content.
Rules / edge behavior	An admin cannot delete their own account. The final enabled ADMIN is protected. Foreign-key dependencies result in 409.
Java implementation	controller/AdminUserController.java:57-65; service/AdminUserService.java:86-106; repository/UserRepository.java:21-23

8 Admin site management

GET /api/admin/sites

Access	ADMIN bearer token
Purpose	List solar installations for operations staff.
Input	Query: page, size (max 100).
Success	200 OK paginated, sorted by name.
Rules / edge behavior	Invalid pagination returns 400.
Java implementation	controller/AdminSiteController.java:31-36; service/AdminSiteService.java:38-42; repository/SolarSiteRepository.java:18-19

GET /api/admin/sites/{id}

Access	ADMIN bearer token
Purpose	Inspect one site and its owner metadata.
Input	Path: site ID.
Success	200 OK.
Rules / edge behavior	Missing site returns 404.
Java implementation	controller/AdminSiteController.java:38-41; service/AdminSiteService.java:44-46,113-116

POST /api/admin/sites

Access	ADMIN bearer token
Purpose	Create the ownership boundary for devices, batteries, telemetry, and alerts.
Input	JSON: ownerId, unique code, name, optional address, positive capacityKw, active.
Success	201 Created.
Rules / edge behavior	Owner must exist and have HOMEOWNER role. Codes are trimmed and uppercased. Duplicate code is 409.
Java implementation	controller/AdminSiteController.java:43-46; service/AdminSiteService.java:49-65,104-111; dto/site/SiteRequest.java:14-35

PUT /api/admin/sites/{id}

Access	ADMIN bearer token
Purpose	Maintain site metadata or transfer an unused site.
Input	Same JSON shape as create.
Success	200 OK.
Rules / edge behavior	Ownership cannot change while devices, batteries, telemetry, storage history, or alerts depend on the site; returns 409.
Java implementation	controller/AdminSiteController.java:48-51; service/AdminSiteService.java:67-85,96-102

DELETE /api/admin/sites/{id}

Access	ADMIN bearer token
Purpose	Remove a site that has never accumulated operational or historical dependencies.
Input	Path: site ID.
Success	204 No Content.
Rules / edge behavior	Protected history is never cascaded away; any dependency causes 409.
Java implementation	controller/AdminSiteController.java:53-57; service/AdminSiteService.java:87-102

9 Admin device management

GET /api/admin/devices

Access	ADMIN bearer token
Purpose	Search the device inventory.
Input	Query: page, size, optional status, type, siteld.
Success	200 OK paginated by identifier.
Rules / edge behavior	Enum values are strict. Device types: SOLAR_INVERTER, SMART_METER, BATTERY_CONTROLLER, WEATHER_SENSOR.
Java implementation	controller/AdminDeviceController.java:33-41; service/AdminDeviceService.java:35-49; repository/DeviceRepository.java:16-19

GET /api/admin/devices/{id}

Access	ADMIN bearer token
Purpose	Inspect one device plus site and owner references.
Input	Path: device ID.
Success	200 OK.
Rules / edge behavior	Missing device returns 404.
Java implementation	controller/AdminDeviceController.java:43-46; service/AdminDeviceService.java:51-54,108-118

POST /api/admin/devices

Access	ADMIN bearer token
Purpose	Register a physical or logical monitoring device.
Input	JSON: identifier, name, type, status, siteld, optional manufacturer/model/firmwareVersion/lastSeenAt.
Success	201 Created.
Rules / edge behavior	Identifier is unique and normalized uppercase. Site must exist and belong to a HOMEOWNER.
Java implementation	controller/AdminDeviceController.java:48-51; service/AdminDeviceService.java:56-64,91-105; dto/device/DeviceRequest.java:16-45

PUT /api/admin/devices/{id}

Access	ADMIN bearer token
Purpose	Update lifecycle state and descriptive metadata.
Input	Same JSON shape as create.
Success	200 OK.
Rules / edge behavior	A device cannot move to another site after telemetry or alerts reference it; this preserves historical meaning.
Java implementation	controller/AdminDeviceController.java:53-56; service/AdminDeviceService.java:67-80,91-105

DELETE /api/admin/devices/{id}

Access	ADMIN bearer token
Purpose	Remove an unused device record.
Input	Path: device ID.
Success	204 No Content.
Rules / edge behavior	Telemetry or alert dependencies produce 409 instead of cascading deletes.
Java implementation	controller/AdminDeviceController.java:58-62; service/AdminDeviceService.java:82-88

10 Admin battery management

GET /api/admin/batteries

Access	ADMIN bearer token
Purpose	List storage assets and their current snapshots.
Input	Query: page, size (max 100).
Success	200 OK paginated.
Rules / edge behavior	Invalid pagination returns 400.
Java implementation	controller/AdminBatteryController.java:31-36; service/AdminBatteryService.java:32-36; repository/BatteryRepository.java:14-16

GET /api/admin/batteries/{id}

Access	ADMIN bearer token
Purpose	Inspect one battery and its site/owner.
Input	Path: battery ID.
Success	200 OK.
Rules / edge behavior	Missing battery returns 404.
Java implementation	controller/AdminBatteryController.java:38-41; service/AdminBatteryService.java:38-41,97-107

POST /api/admin/batteries

Access	ADMIN bearer token
Purpose	Register a battery and its current capacity/charge snapshot.
Input	JSON: identifier, name, siteId, capacityKwh, status, currentChargePercent, currentStoredEnergyKwh, lastUpdatedAt.
Success	201 Created.
Rules / edge behavior	Identifier is unique. Charge is 0-100. Stored energy cannot exceed capacity. Site must be a homeowner site.
Java implementation	controller/AdminBatteryController.java:43-46; service/AdminBatteryService.java:43-51,78-95; dto/storage/BatteryRequest.java:19-50

PUT /api/admin/batteries/{id}

Access	ADMIN bearer token
Purpose	Update the current battery snapshot or metadata.
Input	Same JSON shape as create.
Success	200 OK.
Rules / edge behavior	History or alerts prevent moving the battery to another site.
Java implementation	controller/AdminBatteryController.java:48-51; service/AdminBatteryService.java:54-67,78-95

DELETE /api/admin/batteries/{id}

Access	ADMIN bearer token
Purpose	Remove an unused battery record.
Input	Path: battery ID.
Success	204 No Content.
Rules / edge behavior	Storage history or alerts produce 409; historical records are preserved.
Java implementation	controller/AdminBatteryController.java:53-57; service/AdminBatteryService.java:69-75

11 Admin settings and alerts

GET /api/admin/settings

Access	ADMIN bearer token
Purpose	Read the singleton rules that drive background alert generation.
Input	No body.
Success	200 OK with thresholds and feature flags.
Rules / edge behavior	Creates defaults if the singleton row does not yet exist.
Java implementation	controller/AdminSystemSettingController.java:20-23; service/SystemSettingService.java:18-21, 49-67

PUT /api/admin/settings

Access	ADMIN bearer token
Purpose	Configure production, consumption, battery, offline, and email alert behavior.
Input	JSON with three thresholds, four alert toggles, offline threshold minutes, and emailNotificationsEnabled.
Success	200 OK.
Rules / edge behavior	Low battery is 0-100; thresholds are nonnegative; offline minutes are 1-10080. Settings are rules, not alert events.
Java implementation	controller/AdminSystemSettingController.java:25-28; service/SystemSettingService.java:23-36; dto/setting/SystemSettingRequest.java:15-39

GET /api/admin/alerts

Access	ADMIN bearer token
Purpose	Monitor persisted alert events across all homeowners.
Input	Query: optional type, severity, read, userId; page, size.
Success	200 OK paginated newest first.
Rules / edge behavior	Strict enums and bounded pagination. Optional filters are combined with JPA Specifications.
Java implementation	controller/AdminAlertController.java:24-33; service/AlertService.java:42-59; repository/AlertRepository.java:16-19

12 Homeowner profile and sites

GET /api/homeowner/profile

Access	HOMEOWNER bearer token
Purpose	Return the authenticated homeowner's own identity record.
Input	No body.
Success	200 OK.
Rules / edge behavior	User ID comes from AuthenticationPrincipal, never from a client-supplied path or query value.
Java implementation	controller/HomeownerController.java:26-29; service/HomeownerService.java:25-28, 57-65

PUT /api/homeowner/profile

Access	HOMEOWNER bearer token
Purpose	Let a homeowner maintain name, email, and phone.
Input	JSON: name, email, optional phone.
Success	200 OK.
Rules / edge behavior	Duplicate email is 409. Changing email invalidates existing JWT subjects, so the client must log in again.
Java implementation	controller/HomeownerController.java:31-36; service/HomeownerService.java:30-41; dto/homeowner/UpdateProfileRequest.java:11-23

GET /api/homeowner/sites

Access	HOMEOWNER bearer token
Purpose	List only the current homeowner's solar sites.
Input	No body.
Success	200 OK array; empty array is valid.
Rules / edge behavior	Repository query includes ownerId; filtering is done in SQL, not in frontend code.
Java implementation	controller/HomeownerController.java:38-41; service/HomeownerService.java:43-48; repository/SolarSiteRepository.java:24-25

GET /api/homeowner/sites/{id}

Access	HOMEOWNER bearer token
Purpose	Read an owned site without IDOR exposure.
Input	Path: site ID.
Success	200 OK.
Rules / edge behavior	Query matches both id and ownerId. A foreign site returns 404, hiding its existence.
Java implementation	controller/HomeownerController.java:43-46; service/HomeownerService.java:50-55; repository/SolarSiteRepository.java:27-28

13 Homeowner energy and reporting

GET /api/homeowner/energy/summary

Access	HOMEOWNER bearer token
Purpose	Aggregate production, consumption, grid import/export, and reading count.
Input	Query: optional siteld, from, to (ISO-8601 instants). Defaults to the last 24 hours.
Success	200 OK; zero totals are valid.
Rules / edge behavior	from must precede to; max range 366 days; supplied site must be owned. Ownership is included in the aggregate query.
Java implementation	controller/HomeownerEnergyController.java:28-35; service/HomeownerEnergyService.java:29-37,52-68; repository/EnergyReadingRepository.java:27-42

GET /api/homeowner/energy/readings

Access	HOMEOWNER bearer token
Purpose	Retrieve raw domain readings for client-side charts or analysis.
Input	Query: optional siteld/from/to; page (default 0); size (default 50, max 200).
Success	200 OK paginated newest first.
Rules / edge behavior	Ownership and time range are part of the repository query. No unbounded telemetry load occurs.
Java implementation	controller/HomeownerEnergyController.java:37-46; service/HomeownerEnergyService.java:39-49; repository/EnergyReadingRepository.java:19-25

GET /api/homeowner/reports/energy

Access	HOMEOWNER bearer token
Purpose	Return daily aggregates using meaningful parameterized JDBC.
Input	Query: optional siteld, from, to (ISO dates). Defaults to last 30 UTC dates; max 366 days.
Success	200 OK array grouped by date and site.
Rules / edge behavior	Site ownership is checked before the query and enforced again in SQL using ownerId. SQL parameters are never concatenated from user input.
Java implementation	controller/HomeownerReportController.java:26-33; service/EnergyReportService.java:25-51; repository/EnergyReportJdbcRepository.java:16-57

14 Homeowner storage and alerts

GET /api/homeowner/storage/status

Access	HOMEOWNER bearer token
Purpose	Show current state for batteries attached to owned sites.
Input	No body.
Success	200 OK array; empty is valid.
Rules / edge behavior	Repository derives site.owner.id in SQL and entity graph avoids N+1 lookups.
Java implementation	controller/HomeownerStorageController.java:29-32; service/HomeownerStorageService.java:31-36; repository/BatteryRepository.java:24-25

GET /api/homeowner/storage/history

Access	HOMEOWNER bearer token
Purpose	Retrieve time-series charge and stored-energy history.
Input	Query: optional batteryId/from/to; page; size (max 200). Defaults to seven days.
Success	200 OK paginated newest first.
Rules / edge behavior	Max 366 days. A foreign battery returns 404. Owner predicate remains in the reading query.
Java implementation	controller/HomeownerStorageController.java:34-43; service/HomeownerStorageService.java:38-62; repository/StorageReadingRepository.java:16-22

GET /api/homeowner/alerts

Access	HOMEOWNER bearer token
Purpose	List persisted alert events belonging to the current homeowner.
Input	Query: optional read; page; size (max 100).
Success	200 OK paginated newest first.
Rules / edge behavior	User ID always comes from the JWT principal; entity graph loads related references efficiently.
Java implementation	controller/HomeownerAlertController.java:26-33; service/AlertService.java:22-29; repository/AlertRepository.java:27-31

PUT /api/homeowner/alerts/{id}/read

Access	HOMEOWNER bearer token
Purpose	Acknowledge an owned alert.
Input	Path: alert ID; no body.
Success	200 OK with the alert's read flag true.
Rules / edge behavior	Lookup matches id and userId. A foreign or missing alert returns 404. Repeating the request is idempotent in effect.
Java implementation	controller/HomeownerAlertController.java:35-38; service/AlertService.java:31-40; repository/AlertRepository.java:33-34

15 Errors, validation, and status codes

Every API error uses the same basic JSON envelope: timestamp, status, error, message, path, and optional fieldErrors. The advice layer maps known exceptions and hides internal stack traces.

```
{
  "timestamp": "2026-10-01T14:36:22Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/admin/users",
  "fieldErrors": {"email": ["must be a well-formed email address"]}
}
```

Status	Meaning in Solaris
400	Malformed JSON, validation, invalid enum/query/path type, invalid date range
401	Missing/invalid/expired access token, invalid/revoked/replayed refresh token
403	Authenticated but wrong role, or prohibited admin self-delete
404	Missing resource or foreign homeowner-owned resource
405	HTTP method not supported
409	Duplicate unique value, protected dependency, final-admin rule, optimistic lock
503	Redis or authentication data temporarily unavailable
500	Only genuinely unexpected server failures; generic message returned, details logged

```
exception/GlobalExceptionHandler.java:27-156
security/SecurityErrorWriter.java:21-52
```

16 Background monitoring and persisted alerts

SystemSetting rows configure rules; Alert rows record events. This is analogous to keeping Django settings/model configuration separate from a Notification or Alert model.

- @Scheduled invokes monitoring at a configurable fixed delay; no uncontrolled loop or manually managed thread is used.
- A pessimistic lock on the singleton settings row serializes evaluation across application instances.
- Offline device, low battery, high consumption, and low production rules create persisted alerts.
- Repository exists checks enforce a one-hour cooldown per user/type/resource to reduce duplicate alerts.
- The scheduled wrapper catches and logs RuntimeException so one failed run does not terminate future scheduling.

```
service/AlertMonitoringJob.java:9-23  
service/AlertMonitoringService.java:26-130  
repository/SystemSettingRepository.java:10-12  
repository/AlertRepository.java:36-43
```

17 Why one report uses JDBC

Most CRUD uses JPA because entity relationships and transactional updates are clearer there. The daily report uses `NamedParameterJdbcTemplate` because it is an aggregation query whose SQL shape is itself the domain logic: group by date and site, sum four measures, and count readings.

Safety properties

- Named parameters bind `ownerId`, `siteId`, and timestamps; client input is never concatenated into SQL.
- The owner predicate is inside SQL, so even a coding mistake above the repository does not expose another owner's rows.
- The service validates the site and limits the requested date range to 366 days.
- The result maps directly into `DailyEnergyReportResponse` rather than pretending an aggregate row is a JPA entity.

```
service/EnergyReportService.java:25-51  
repository/EnergyReportJdbcRepository.java:13-57
```

18 Postman workflow

Import `output/api/Solaris.postman_collection.json`. It contains all 38 unique routes and 39 runnable requests because login appears once for ADMIN and once for HOMEOWNER.

Recommended order

- Set `baseUrl` and the four credential variables. Use an existing controlled ADMIN; public registration cannot create one.
- Run Admin Login and Homeowner Login. Their test scripts save access tokens, refresh tokens, and `homeownerUserId`.
- Create a site, device, and battery. The collection saves each returned ID for later requests.
- Run homeowner reads only after the admin has associated resources with that homeowner.
- Refresh rotates and overwrites the homeowner token variables. Logout clears them.
- Delete in child-to-parent order: device/battery, site, then user, and only when no telemetry/history dependencies exist.

Telemetry ingestion note

This backend exposes telemetry read APIs but no public/admin ingestion endpoint in the current route set. `EnergyReading` and `StorageReading` records are expected to arrive from a trusted integration or a future ingestion module. The live suite seeded those records directly in PostgreSQL only as disposable test fixtures.

19 Source map and maintenance notes

Concern	Location	Responsibility
HTTP controllers	controller/*.java	Route, HTTP status, DTO binding, principal extraction
Request/response contracts	dto/**/*.java	JSON fields, validation, password exclusion
Business logic	service/*.java	Transactions, ownership, lifecycle and conflict rules
Persistence	repository/*.java	JPA derived queries, Specifications, entity graphs, JDBC report
Database model	entity/*.java	Tables, foreign keys, indexes, uniqueness, timestamps, optimistic versions
Security	config/SecurityConfig.java; security/*.java	Route roles, JWT filter, token generation, Redis token state
Errors	exception/*.java	Consistent client-safe JSON errors
Runtime config	resources/application.yaml	PostgreSQL, Redis, JWT TTL, Flyway, monitoring interval
Automated tests	src/test/java/com/solaris/backend/**	44 service, controller/security, repository/JDBC assertions
Live suite	scripts/live_api_e2e.py	127 real HTTP checks with isolated fixture cleanup

When adding an endpoint, preserve this chain: DTO validation -> thin controller -> transactional service rule -> ownership-aware repository query -> DTO response -> positive and negative tests. If any layer is skipped, the API usually becomes harder to secure or maintain.